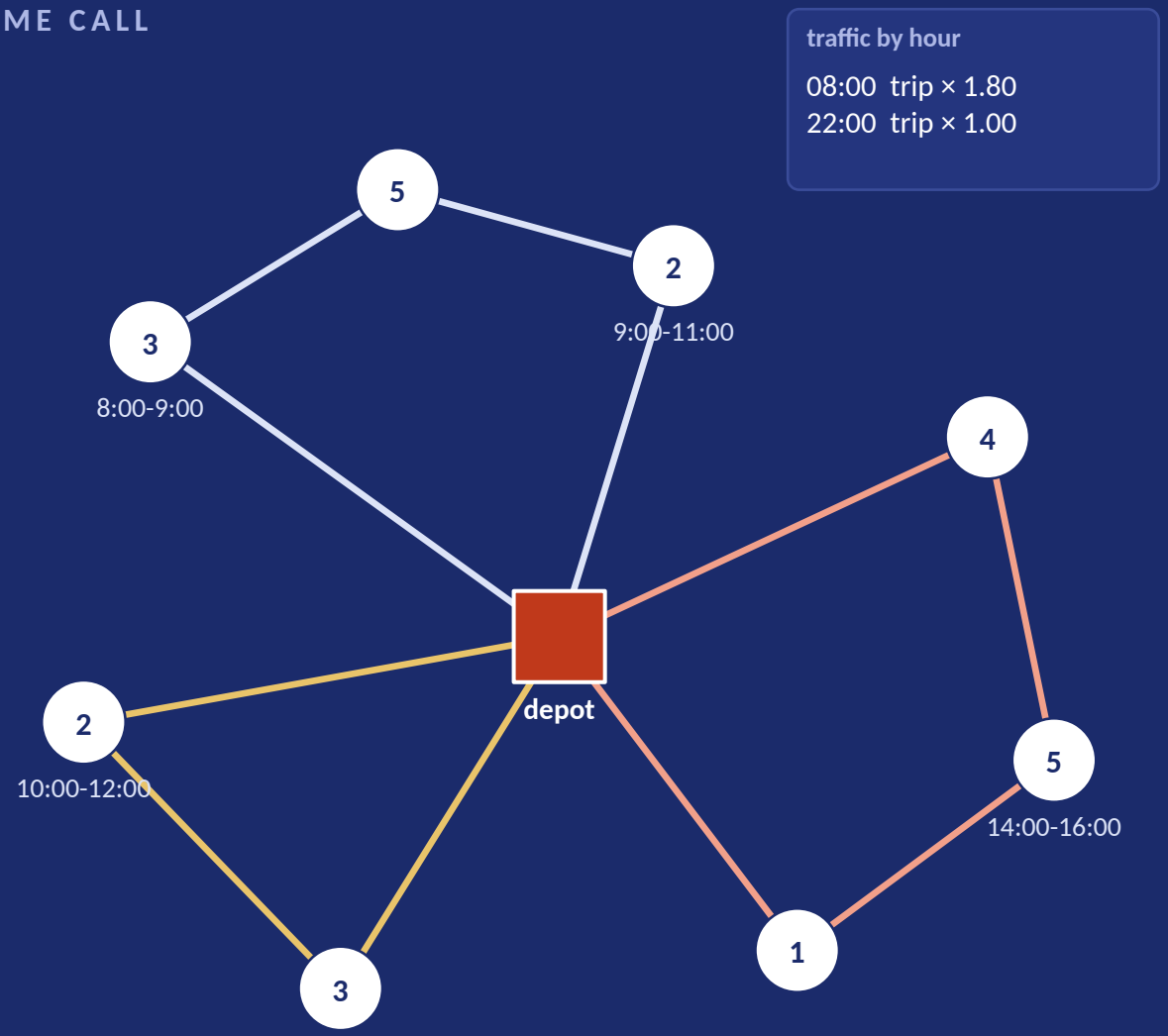
Routing deliveries when the clock matters

THE QUESTION

Which truck serves each customer, in which order, and when does it travel, so that the total route duration is minimal?

Not a shortest path between two points: we decide the assignment, the order and the timing for a whole fleet, under capacity, time windows and traffic.

Pablo · Dag · Farida 13 November 2026



Three layers: capacity, then time windows, then hourly traffic

CVRP — **Capacitated Vehicle Routing Problem**

Visit every customer once; each truck carries at most Q. Level B of the brief.

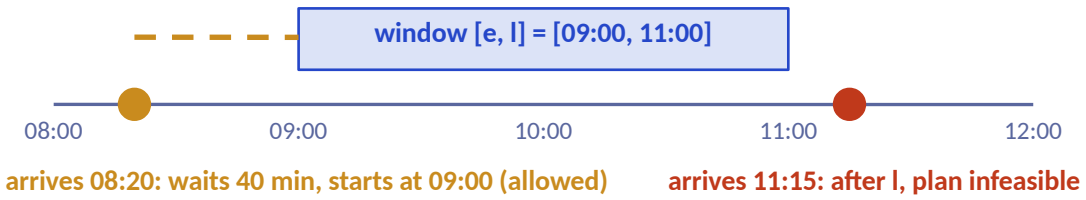
CVRPTW — **CVRP with Time Windows**

Each customer can only be served inside its window [e, l]. Level A of the brief.

+ traffic — **Our extension (ADEME call)**

A trip lasts longer or shorter depending on the hour the truck leaves.

A time window, worked example: customer open from 09:00 to 11:00



arrives 08:20: waits 40 min, starts at 09:00 (allowed)

arrives 11:15: after l, plan infeasible

Q — **Capacity of a truck**

a truck carries at most Q units; a customer i asks $q_i \le Q$

[e_i, l_i] — **Time window of customer i**

earliest and latest start of service; early: wait, late: forbidden

ρ(h) — **Traffic factor of hour h**

a trip leaving at hour h takes ρ(h) × its free-flow time

z — **Objective: total route duration**

sum over trucks of (return time − departure time): driving, waiting and service

Two different ideas

Time window: WHEN a customer can be served.

Traffic: HOW LONG a trip takes, depending on WHEN it leaves.

Our first objective could not see the clock

Version 1: sum of fixed arc times

$$\min \; z = \sum_{(i,j) \in A} c_{ij}\, x_{ij}$$

x_ij = 1 if a truck drives from i to j; c_ij = one fixed number per pair of places.

Trip A → B, free-flow 10 min	leave 08:00	leave 22:00
Version 1 counts	10 min	10 min
With hourly traffic	18 min	10 min

VERSION 2 | VALIDATED

$$\min \; z = \sum_{k \in K} \left(t_{n+1}^{k} - t_{0}^{k} \right)$$

For each truck k: time back at the depot (t_{n+1}^k) minus time it left (t_0^k), added over the trucks. One component, exactly the brief's "total route duration": driving, waiting and service are all inside it.

01

The hour is invisible

With a fixed c_ij, the 08:00 peak and 22:00 give the same z: traffic, the point of the ADEME call, had no effect.

Version 2: each trip is priced at the hour it starts (next slide)

02

It was only distance

At constant speed the sum of arc times is the distance: the textbook CVRP, not the brief, which asks for the total route duration.

Version 2: the target is time on the road, not km

03

Waiting was free

A truck arriving at 08:20 at a customer who opens at 09:00 waits 40 min. The driver is paid, yet Version 1 counted 0.

Version 2: the 40 min push the return time later, so they are in z

How the hourly traffic matrix is built

1 Road network
streets of the city (OpenStreetMap)

2 Shortest paths
best road route between every pair of places

3 Base matrix C = [c_ij]
free-flow time from i to j (empty roads)

4 Traffic factor ρ(h)
one number per hour of the day (our assumption)

5 Hourly matrix
$c_{ij}^h = \rho(h) \times c_{ij}$

Real road-network matrices (steps 1 to 3) + assumed hourly traffic profile (step 4, the same for Paris and São Paulo)

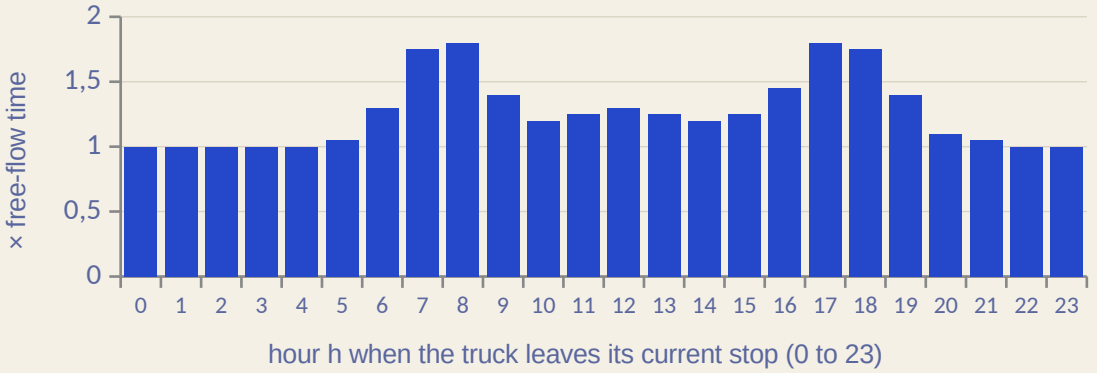
Worked example: free-flow time A → B = 10 min

Truck leaves A at	ρ(h)	Trip A → B takes
08:00 (peak)	1.80	**18 min**
14:00	1.20	**12 min**
22:00 (night)	1.00	**10 min**

Each leg uses the hour the truck leaves its current stop, not the hour it left the depot: a route starting at 07:40 can drive its first leg at ρ(7) = 1.75 and a later leg at ρ(9) = 1.40.

In the exact model: $a_j = t_i + s_i + \rho(h)\, c_{ij}$ for the hour h with $y_{ih} = 1$. **y_{ih}** is a yes/no variable: 1 if the truck leaves customer i during hour h. It picks which hourly matrix prices the next trip (t_i service start, s_i service time, a_j arrival).

Assumed traffic factor ρ(h)



1.80 at 08:00 and 17:00: rush hours, a trip takes 80 % longer. **1.00 from 22:00 to 04:59:** empty roads, free-flow time. The values are our assumption, not measured congestion; measured speeds would change the numbers, not one equation.

The problem is NP-hard, and the exact model stops at a few customers

NP-hard, in our words: no known algorithm solves every instance in a time that grows polynomially with n. For an exact solver, each extra customer multiplies the number of possible plans.

Hamiltonian cycle

a cycle through every vertex once?
NP-complete (Karp 1972)



built in $O(m^2)$

Our problem, decision version

is there a plan with $z \leq B$?

The construction, from a graph with vertices u_0 ... u_m

- one truck (K = 1), demand 1 each, Q = m; windows = the whole day; ρ = 1
- travel time 1/(m+1) on an edge of the graph, 2/(m+1) on a non-edge
- threshold B = 1

A tour has m + 1 legs. Only edges: duration exactly 1. One non-edge: at least (m+2)/(m+1) > 1. So a plan with z ≤ 1 exists if and only if the graph has a Hamiltonian cycle.

Consequence: exact methods cannot scale; we use the ILP to check small cases and as a reference, and metaheuristics for real sizes.

Where our ILP stops (HiGHS solver, 60 s per solve)

7 to 8 customers	Full model with hourly traffic (duration): proven optimal up to 8 on C101 and R101, 7 on RC101; never at 9 or 10.
100 customers	Time windows, no traffic, distance: proven optimal on C101 and R101 (equal to the best known value).
25 customers	Same setting on the mixed RC101: proven up to 25; from 50 on, 6 to 13 % between plan and bound at 60 s.

Three different settings: the sizes are not comparable with each other. Traffic is what makes the model hard (24 hourly copies of each departure, weak big-M bounds).

Three metaheuristics: how each one searches

Tabu Search

Pablo

1. Start: time-oriented nearest neighbor (go to the closest customer that keeps the route feasible).
2. Sample 200 moves (relocate, swap, 2-opt, 2-opt*) near each customer's 15 closest neighbors: the full neighborhood is too large.
3. Apply the best sampled move that is not tabu, even if it makes the plan worse: this is how it leaves a local optimum.
4. Tabu memory: a moved customer cannot move again for 10 to 30 iterations (tenure 20), so the search does not cycle back.
5. Aspiration: a tabu move is allowed if it gives a new best plan.

Memory, not chance, drives it away from where it has been.

tenure 20 · 200 candidates · 15 neighbors

Simulated Annealing

Dag

1. Start: randomized cheapest insertion (each customer at its cheapest feasible place).
2. Draw one random move among 5: relocate one customer or a chain of 2 to 3, swap, 2-opt, 2-opt*.
3. Δ = new cost − current cost. If Δ ≤ 0 (better or equal), accept.
4. If Δ > 0, accept with probability e^(−Δ/T). Example: Δ = 5 accepted 61 % of the time at T = 10, 0.7 % at T = 1.
5. T (temperature) falls from T_0 to T_f over the time budget: early it explores, at the end it only descends.

Chance, controlled by T, lets it accept worse plans early on.

T_0 = 0.005 × start cost · T_f 0.005 · compiled (Numba)

GRASP-ILS

Farida

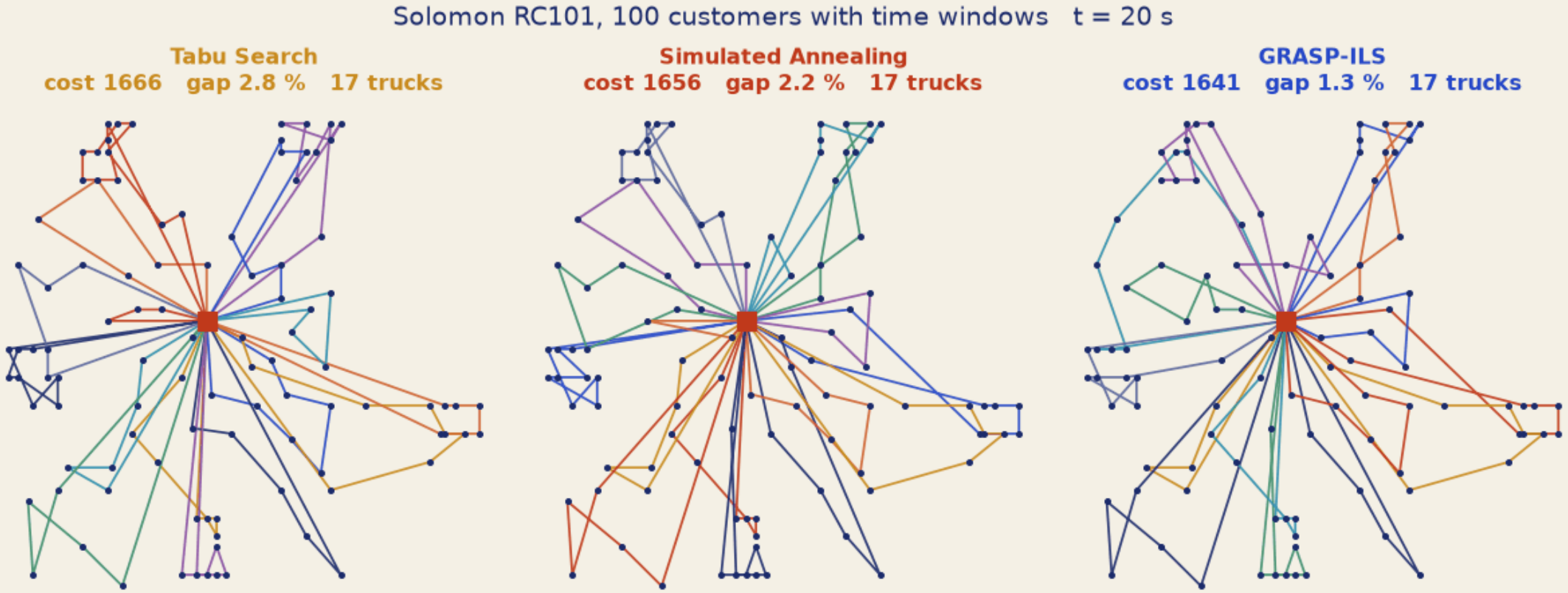
1. Construct (GRASP): regret insertion, serving first customers with few good options; pick at random among the best (α = 0.3).
2. Local search: descent over 6 neighborhoods (relocate, swap, 2-opt*, Or-opt, 2-opt, cross-exchange), granular, first improvement.
3. Kick (ILS): remove a cluster of 10 to 20 % of the customers, reinsert them, descend again; keep only if better.
4. Up to 90 kicks per iteration, then a new construction; keep the best plan over all iterations.
5. Time windows checked in O(1) per move (forward and backward time slack per route).

Good random starts, deep descents, kicks to escape.

α 0.3 · first improvement · 90 kicks (configured)

Same 100 customers, same 20 seconds: three ways to search

Solomon RC101 | 100 customers | time windows | objective: distance | 20 s | seed 1



Red square: the depot

cost: total distance of the plan

Dots: the 100 customers

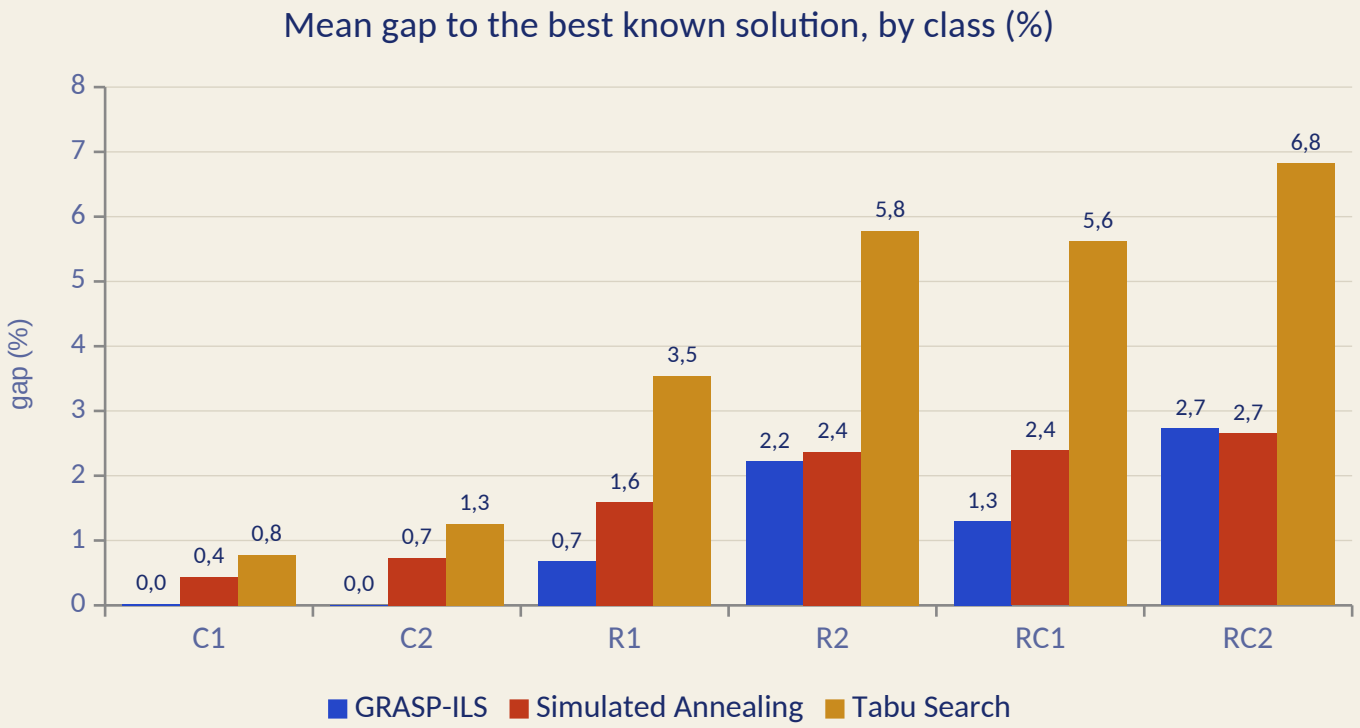
gap: % above the best known value

One color: one truck's route

t = ...: seconds since the start: 0.3, 0.6, 1, 2, 4, 8, 14, 20

Round 1: GRASP-ILS wins on the 50 Solomon test instances

Solomon | 50 test instances | 100 customers | time windows | objective: distance | 10 seeds × 20 s | vs best known



Classes: C clustered customers, R random, RC mixed. Suffix 1: tight windows, short day, small trucks; suffix 2: wide windows, long day, large trucks.
Gap (%) = (our cost − best known) / best known × 100. Best known = best published plan (distance); not a lower bound.

1.16 %
GRASP-ILS
worst run 9.8 % · feasible 500 / 500

1.70 %
Simulated Annealing
worst run 6.5 % · feasible 500 / 500

3.98 %
Tabu Search
worst run 14.0 % · feasible 500 / 500

Is the difference real or luck?
Friedman test (do the three differ at all?): $p = 4 \cdot 10^{-13}$, yes.
Wilcoxon test per pair, Holm correction: every pair differs at 5 %.
Choice rule fixed before the runs: feasible on every run, then lowest mean gap: GRASP-ILS.

“100 customers on Euclidean points is not the real world. Show me the limit of each algorithm.”

Round 1 answered “which algorithm is best on the benchmark”. Round 2 sets six new objectives:

1 Larger instances

400 to 10 000 customers, not only 100

slides 11 to 13

2 Real road networks

real streets in two countries, not straight lines

slides 10, 14

3 Scalability

where each algorithm breaks, and why

slides 11 to 13

4 Parameter sensitivity

how fragile each tuned setting is

slide 16

5 Departure time

what the hour does to route duration

slide 15

6 A recommendation

which algorithm, in which scenario

slide 17

A new test bed: each dataset answers one question

Dataset	Customers	Windows	Objective	Reference	What it tests
Solomon (round 1)	100	yes	distance	best known	quality, statistics
Homberger R1	400 · 800 · 1 000	yes	distance	best known	scale, windows
Synthetic uniform	1 000 to 10 000	no	distance	lower bound	computing limit
Paris (France)	999 real	no	distance + duration	best of our runs	real roads
São Paulo (Brazil)	999 real	no	distance + duration	best of our runs	second country

Time windows were tested up to 1 000 customers. The 10 000-customer run has no time windows.

Why distance on benchmarks? Published reference values are distances. Same solvers; only the route cost function changes (distance, or duration with ρ(h)). Duration with traffic is run in the cities (slide 15).
Cities: RRNCO dataset, built on OpenStreetMap. Real road network; distances and free-flow times are shortest paths on the real graph. The hourly factor ρ(h) is our assumed profile, the same in both cities.

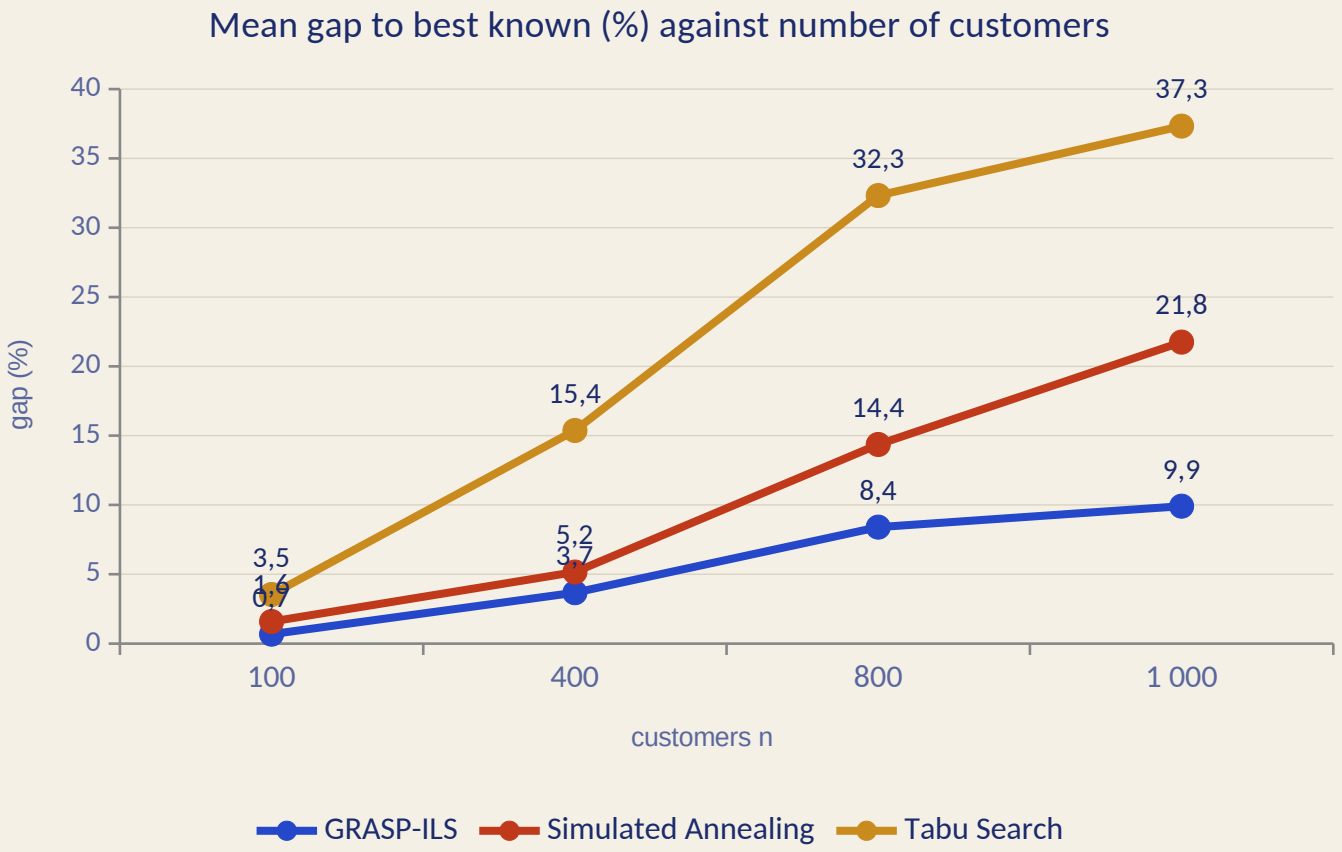
Road-network characteristics

	Paris	S. Paulo
Mean speed	**21 km/h**	**32 km/h**
free-flow km ÷ free-flow hours, all pairs		
Circuity	**1.51**	**1.65**
road distance ÷ straight-line distance (median, pairs over 50 m apart)		
Asymmetry	**97 %**	**99 %**
share of pairs where A → B and B → A differ by more than 10 m (one-way streets)		
Area	**8.9 km²**	**8.7 km²**
bounding box of the 1 000 points		

Computed from the matrices; not traffic measurements.

As n grows, every algorithm loses quality, but not at the same speed

Homberger R1 (100: Solomon R1) | time windows | objective: distance | 20 s | 3 seeds | vs best known



Not the same scenario as the 10 000-customer run (slide 13): here every customer has a time window.

GRASP-ILS 9.9 %

Best at every size. At 1 000 it completes a single iteration in 20 s: its regret construction alone gives the best start.

Simulated Annealing 21.8 %

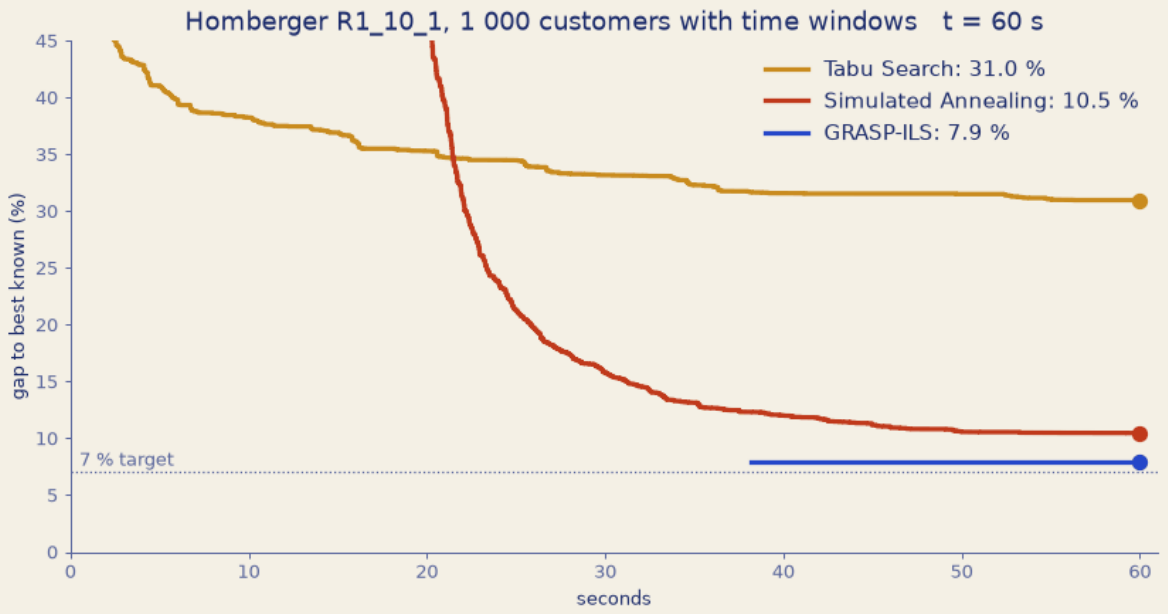
16 million moves in 20 s are only 16 000 per customer: not enough exploration yet (slide 12).

Tabu Search 37.3 %

175 trucks against 107 for GRASP-ILS. Its start opens too many routes and its moves rarely empty one.

More time helps SA most; GRASP-ILS stays ahead at every budget

Homberger R1 | 1 000 customers | time windows | objective: distance | 20 / 60 / 120 s | vs best known



Animation: one run per algorithm (60 s); best gap so far against seconds. Dashed line: 7 % target.

Why three budgets? To tell apart “not enough time” from “a limit of the method”: if more time closes the gap, time was the problem.

Mean gap, 3 seeds	20 s	60 s	120 s
GRASP-ILS	9.9 %	9.0 %	7.8 %
Simulated Annealing	21.8 %	15.0 %	10.4 %
Tabu Search	37.3 %	33.8 %	31.7 %

GRASP-ILS ahead at every budget

9.9 % → 7.8 %: one or two iterations fit, so time helps little.

SA gains the most

21.8 % → 10.4 %, divided by 2.1: it was short of time.

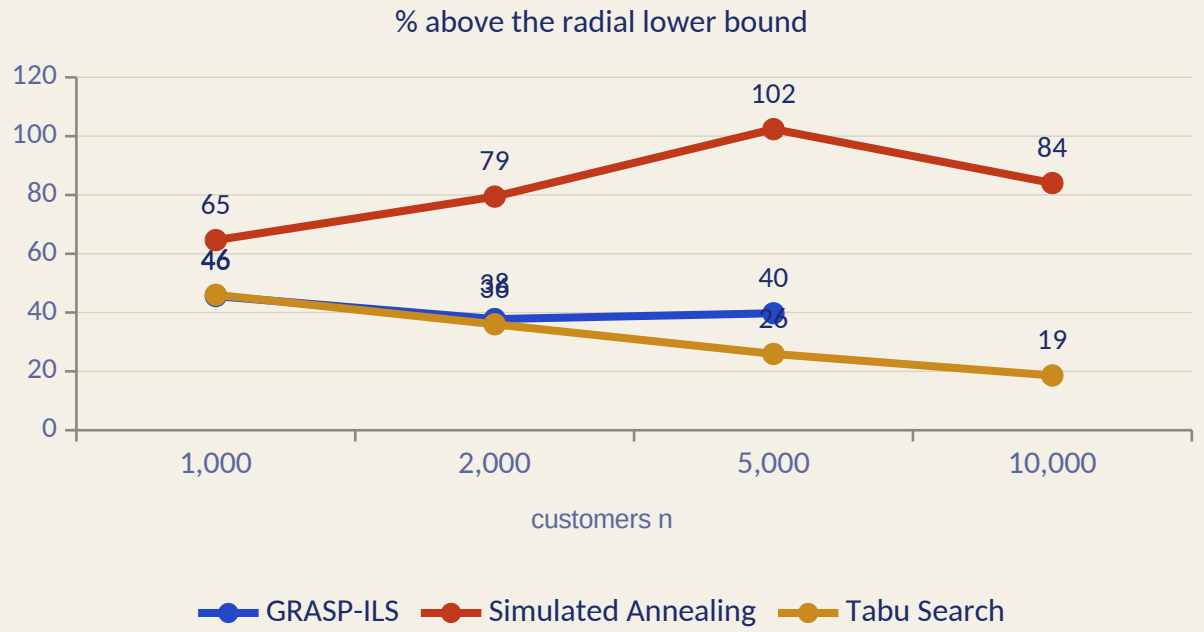
Tabu barely moves

37.3 % → 31.7 %: its problem is the trucks.

7 % target: the brief caps the grade above a 7 to 10 % gap; we use the strict 7 %. GRASP-ILS meets it at 800 customers with 120 s (6.7 %), not at 1 000 (7.8 %).

Up to 10 000 customers without windows: who still answers?

synthetic uniform | 1 000 to 10 000 customers | NO time windows | objective: distance | 60 s (10 000: 300 s) | 1 seed | vs radial lower bound



RADIAL LOWER BOUND

$$LB = \frac{2}{Q} \sum_{i=1}^{n} d_{0i}\, q_i$$

d_0i depot-customer distance, q_i demand, Q capacity. A truck must reach its farthest customer and come back, and carries at most Q, so no plan can cost less.

LB is not a plan and not the optimum: % above LB is not a gap to a best known value.

Tabu Search: 18.6 % above LB

At 10 000: 553 trucks, the minimum. Neighbor lists keep one iteration cheap: 0.7 ms at 1 000, 2.1 ms at 10 000.

Simulated Annealing: 84 % above LB

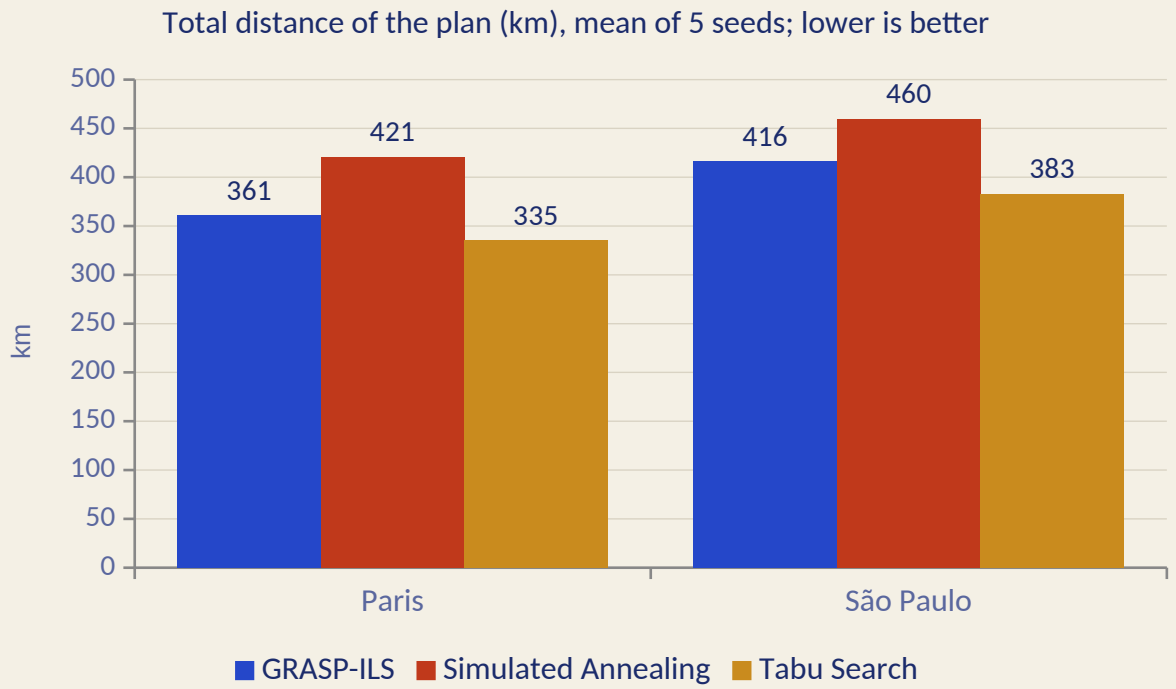
At 10 000: 774 trucks. 14 million moves are 1 400 per customer: its cooling ends long before the plan is good.

GRASP-ILS: no plan

One iteration fills the budget: 63 s at 2 000, 125 s at 5 000 (budget 60 s). At 10 000 the observed run passed 684 s on a 300 s budget without a plan; we stopped it.

On real roads without windows, Tabu Search wins

RRNCO / OpenStreetMap | 999 real customers | NO time windows | objective: distance (km) | 60 s | 5 seeds



Why km here, if the model minimizes duration? SA's solver has the distance objective only, so all three compete on km. With the duration objective and traffic (Tabu and GRASP-ILS, leaving at 08:00) the order is the same: Paris 1,831 vs 2,390 min, São Paulo 1,511 vs 1,727 min.

No windows: construction matters less

Any truck can take any customer. What counts is the number of improving moves: Tabu makes about 30 000 iterations in 60 s, GRASP-ILS one.

GRASP-ILS never reaches its kicks

Run with 0, 10 or 30 kicks: identical result (357.5 km in Paris). The first construction and descent fill the 60 s.

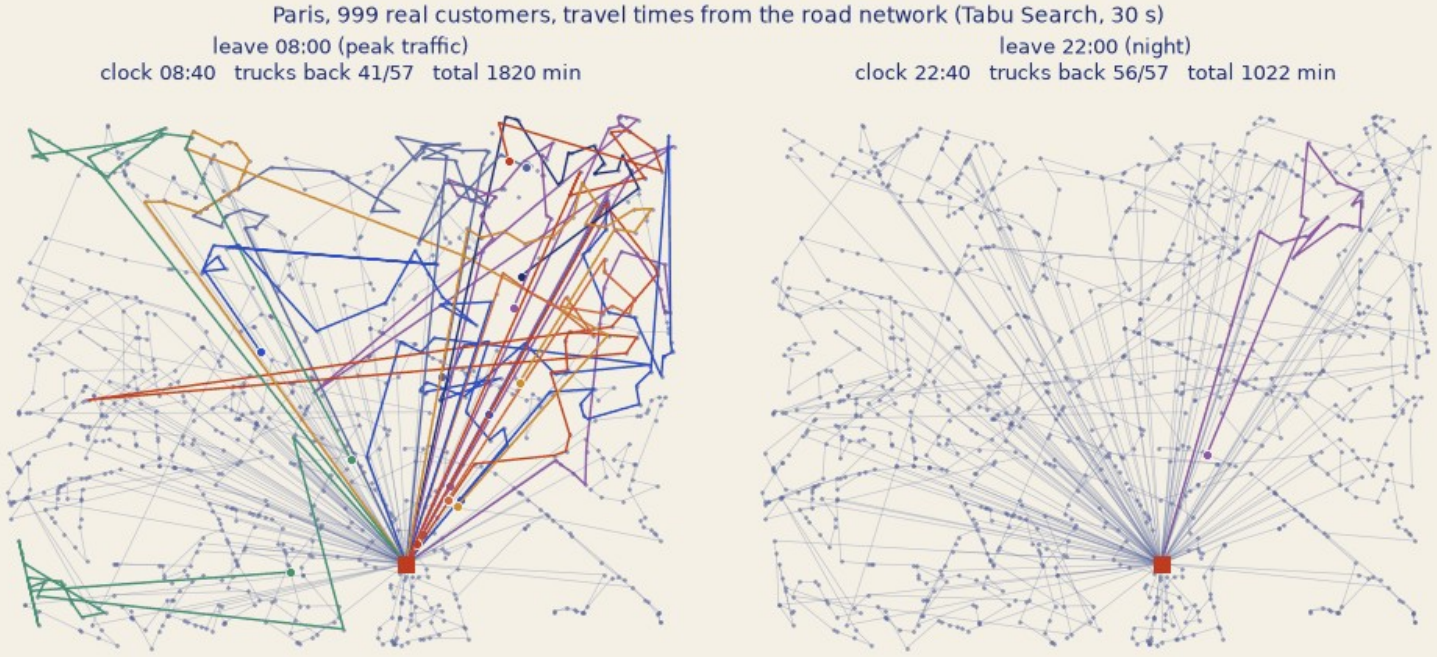
Five times the time: same order

300 s in Paris: Tabu 318 km, GRASP-ILS 345 km. In São Paulo GRASP-ILS does not improve at all.

Takeaway: the best algorithm depends on the scenario. Windows: GRASP-ILS. Large real roads without windows: Tabu Search.

Leaving at 22:00 instead of 08:00: 44.6 % less time on the road

Paris, São Paulo | 999 customers | no time windows | objective: duration with ρ(h) | Tabu Search | 60 s, 5 seeds



Animation: one Tabu Search run per hour (30 s). Same customers, same assumed ρ(h); only the fleet departure hour changes, and each hour is optimized separately.

Paris

−44.6 %

1,831 min at 08:00 → 1,014 min at 22:00

São Paulo

−43.7 %

1,511 min at 08:00 → 850 min at 22:00

$$1 - 1/\rho(8) = 1 - 1/1.80 = 44.4\ \%$$

What it shows: the size of the assumed traffic effect on route duration: the whole saving is the 08:00 factor 1.80.

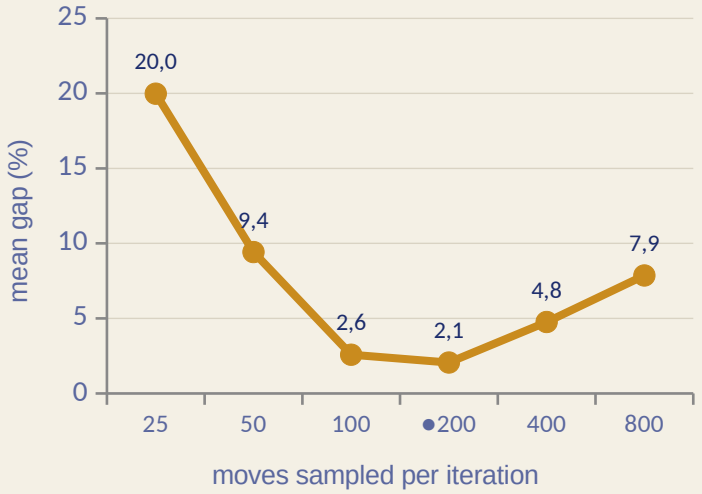
What it does not show: a different visit order at night. Routes last 32 min on average: 1 of 57 changes hour, none crosses midnight. That test needs routes spanning several hours.

Each algorithm has one parameter that can break it

6 tuning instances | 100 customers | time windows | objective: distance | 3 seeds × 20 s | vs best known

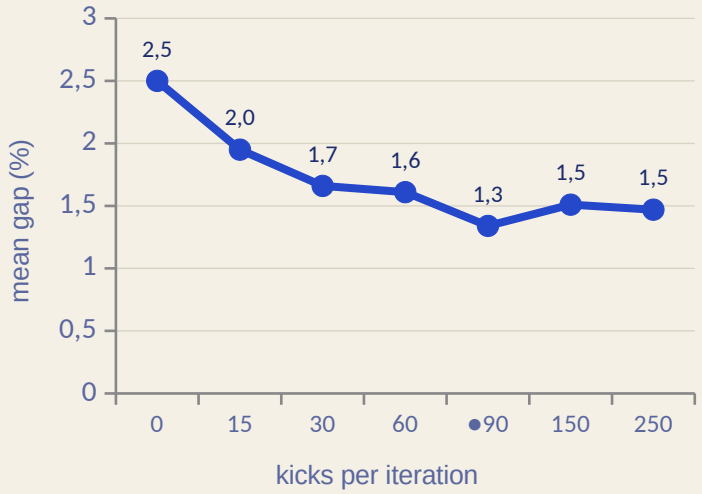
One factor at a time: move one parameter, keep every other at its tuned value, measure the mean gap. It shows how fragile each tuned choice is. ● = value used in the campaign.

Tabu Search: candidate list size



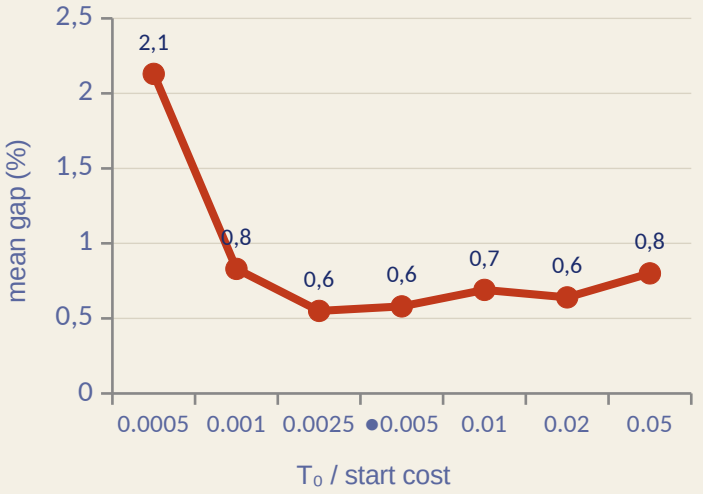
25: 20.0 % (step almost random). 200: 2.1 %. 800: 7.9 % (each step 14 times slower than with 25, so far fewer steps). More is not better.

GRASP-ILS: kicks per iteration



0 kicks (plain GRASP): 2.5 %; 90: 1.3 %. The kicks halve the gap: the ILS part is what makes it win.

SA: initial temperature factor



Flat from 0.001 to 0.05 (0.55 % to 0.83 %). Only near 0 (0.0005: 2.1 %) it breaks: no worse move accepted, a plain descent.

No universal winner: the choice depends on windows and size

For the ADEME pilot **GRASP-ILS** the brief is a CVRPTW: time windows, one depot, a few hundred deliveries.

Scenario	Choose	Because (our data)	Evidence
About 100 customers, time windows	GRASP-ILS	1.16 % mean gap on 500 runs, significantly ahead	Solomon
100 customers, a bad run is not acceptable	Simulated Annealing	best worst case (6.5 %), every run within 7 %; robust to its parameters	Solomon, sensitivity
400 to 1 000 customers, time windows	GRASP-ILS	lowest gap at every size and budget; 7.8 % at 1 000 with 120 s	Homberger
About 1 000 customers, real roads, no windows	Tabu Search	6 % to 7 % from the best plan found, GRASP-ILS 15 % to 17 %	Paris, São Paulo
2 000 to 10 000 customers, no windows	Tabu Search	ahead from 2 000; 18.6 % above the lower bound at 10 000; GRASP-ILS returns no plan	Synthetic

Why no single winner
Windows make the construction decisive: GRASP-ILS's regret start wins. Without windows, improving moves per second decide: Tabu Search's cheap iterations win.

Caveats
We did not test 10 000 customers with time windows (windows stop at 1 000). The switch to Tabu Search above 2 000 customers comes from runs without windows, not from a large CVRPTW.

What we did not solve, and what it means for the results

1 Traffic is assumed

ρ(h) is our profile, not measured hourly data, and it is the same in Paris and São Paulo.

Consequence: Durations and the 44 % night saving are valid under this assumption only. Measured speeds change the matrix, not the model.

2 Equal time is not equal code

SA is compiled (Numba); Tabu Search and GRASP-ILS are pure Python. Same machine, same evaluator, same wall clock.

Consequence: Part of each gap is implementation speed. SA still finishes 2nd on Solomon despite this advantage.

3 One departure time per fleet

The model lets each truck choose its departure; the metaheuristics make all trucks leave at the same time.

Consequence: Our durations can only be equal to or worse than what the model allows: trucks cannot stagger to avoid the peak.

4 First in, first out not guaranteed

Each trip is priced by its departure hour. 60 min free-flow: leave 08:59, ρ 1.80, arrive 10:47; leave 09:00, ρ 1.40, arrive 10:24.

Consequence: Waiting can pay off artificially at an hour boundary. Splitting trips at the full hour removes it.

5 GRASP-ILS overran its budget

It reads the clock only between iterations: at 10 000 customers the run passed 684 s on a 300 s budget with no plan.

Consequence: All reported results use this original code. A fix (clock checks inside construction and descent) is proposed, not run.

6 Brazil instead of Egypt

Validated open road matrices existed for Paris and São Paulo; Cairo needed an OSM extraction we could not validate in time.

Consequence: Two countries, same protocol, but not the pair requested; no graph metrics (density, degree).

IN ONE SLIDE

GRASP-ILS for the time-window ADEME pilot; Tabu Search for very large instances without windows

1.16 %

GRASP-ILS mean gap

Solomon · 100 customers · windows · 20 s · vs best known

7.8 %

GRASP-ILS gap

Homberger · 1 000 customers · windows · 120 s · vs best known

18.6 %

Tabu Search above the lower bound

Synthetic · 10 000 customers · no windows · 300 s · vs radial LB

Three different settings and references: these numbers are not comparable with each other.

What we built

A duration model with hourly traffic, an NP-hardness proof, an exact ILP, and three metaheuristics on one evaluator.

What we learned

Windows reward a good construction (GRASP-ILS); size without windows rewards cheap iterations (Tabu Search); SA is the most robust.

What we recommend

GRASP-ILS for the pilot. Tabu Search without windows at large n. Next: measured traffic, a regret start for Tabu, a faster GRASP-ILS.

Thank you. Questions?